Brunnerne Inc. - Cryptography Writeup

Author: DedseC | Category: Crypto | Difficulty: Hard

Challenge Description: In Brunnerne Inc. we are constantly working to improve our product portfolio and reach still new standards... So we are adjusting it with an inclusion of one of master chef Feistel's classical ingredients adjusted into a custom addition to take it to a new level.

Provided Files:
1. `bake.py` - The encryption script.
2. `baked_pie.txt` - The resulting 256-character ciphertext.
3. `unbaked_pi.txt` - The digits of Pi used for stream generation.

1. Analyzing the Vulnerability

The encryption routine relies on a 100-character custom base alphabet. The process is divided into two distinct cryptographic mechanisms:

- `pie_crypt`: A stream cipher utilizing the digits of Pi for pseudo-random shifts.
- `custom_ingredient`: A purported Feistel network with 16 rounds.

The primary vulnerability lies within the `custom_ingredient` function. Rather than cryptographically securing the text, the 16 rounds linearly transform the blocks. The logic generates predictable states mirroring a Fibonacci-like sequence in modulo 100.

When given a 64-character key, the 256-character final ciphertext is split into four 64-character blocks (B_0, B_1, B_2, and B_3). We can mathematically deduce the relationships.

1.1 Key Recovery

Due to the lack of proper mixing in the initial round function, the first 64-character block output (B_0) is directly equivalent to the key multiplied by 33, modulo 100:

$$B_0 = (33 \times K) \bmod 100$$

Because 33 and 100 are coprime, we can compute the modular inverse of 33 modulo 100, which is 97. Multiplying the known first ciphertext block by 97 instantly reveals the complete 64-character key:

$$K = (97 \times B_0) \bmod 100$$

1.2 Intermediate State Recovery

With the key recovered, we must isolate the original split plaintext chunks, Left (T_L) and Right (T_R). The equations governing the final two ciphertext blocks are:

$$B_2 = (13 \times T_L + 21 \times T_R + 33 \times K) \bmod 100$$

$$B_3 = (21 \times T_L + 34 \times T_R + 54 \times K) \bmod 100$$

By moving the key terms to the left side and solving the system of equations, we calculate the determinant. The determinant is $13 \times 34 - 21 \times 21 = 442 - 441 = 1$. A determinant of 1 means the matrix is trivially invertible without complex modular arithmetic.

The inverse matrix gives us:

$$X = B_2 - 33 \times K$$

$$Y = B_3 - 54 \times K$$

$$T_L = (34 \times X - 21 \times Y) \bmod 100$$

$$T_R = (-21 \times X + 13 \times Y) \bmod 100$$

2. Reversing the Stream Cipher

Once T_L and T_R are recovered and concatenated, the final step is to reverse the `pie_crypt` stream cipher. This cipher utilizes the digits in `unbaked_pi.txt`, keeping state through an index i initiated by the sum of the key's base indices. Since we recovered the key in Step 1, reversing this stream is straightforward by applying a negative shift.

3. The Exploit Payload

The full decryption chain is implemented in the following Python script:

```
import sys

base = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789æøåÆØÅ .,!?-:()
[]/{}=<>+_@^|~%$#&*`“';"
```

```
with open("unbaked_pi.txt") as f:
    pie = f.read()

with open("baked_pie.txt", "r", encoding="utf-8") as f:
    baked = f.read()

b0, b1, b2, b3 = baked[0:64], baked[64:128], baked[128:192], baked[192:256]

# Step 1: Recover the Key
key = "".join(base[(97 * base.index(c)) % 100] for c in b0)

# Step 2: Recover Intermediate Texts
T_L, T_R = "", ""
for i in range(64):
    x = (base.index(b2[i]) - 33 * base.index(key[i])) % 100
    y = (base.index(b3[i]) - 54 * base.index(key[i])) % 100
    T_L += base[(34 * x - 21 * y) % 100]
    T_R += base[(-21 * x + 13 * y) % 100]

text = T_L + T_R

# Step 3: Decrypt pie_crypt
def pie_crypt_decrypt(text: str, key: str) -> str:
    out = ""
    i = sum(base.index(c) for c in key)
    j = 0
    for c in text:
        d1 = int(pie[i % len(pie)])
        i += base.index(key[j % len(key)])
        j += 1
        d2 = int(pie[i % len(pie)])
        i += base.index(key[j % len(key)])
        j += 1
        shift = 10 * d1 + d2
        out += base[(base.index(c) - shift) % len(base)]
    return out

flag = pie_crypt_decrypt(text, key)
print(flag)
```

4. Capture The Flag

Running the payload cleanly extracts the hidden text, yielding our target flag:

brunner{NB!:_Re-using_the_same_key_without_salting-and-hashing_risks_security_of_all_use-instances,_if_one_instance_leaks_info!}